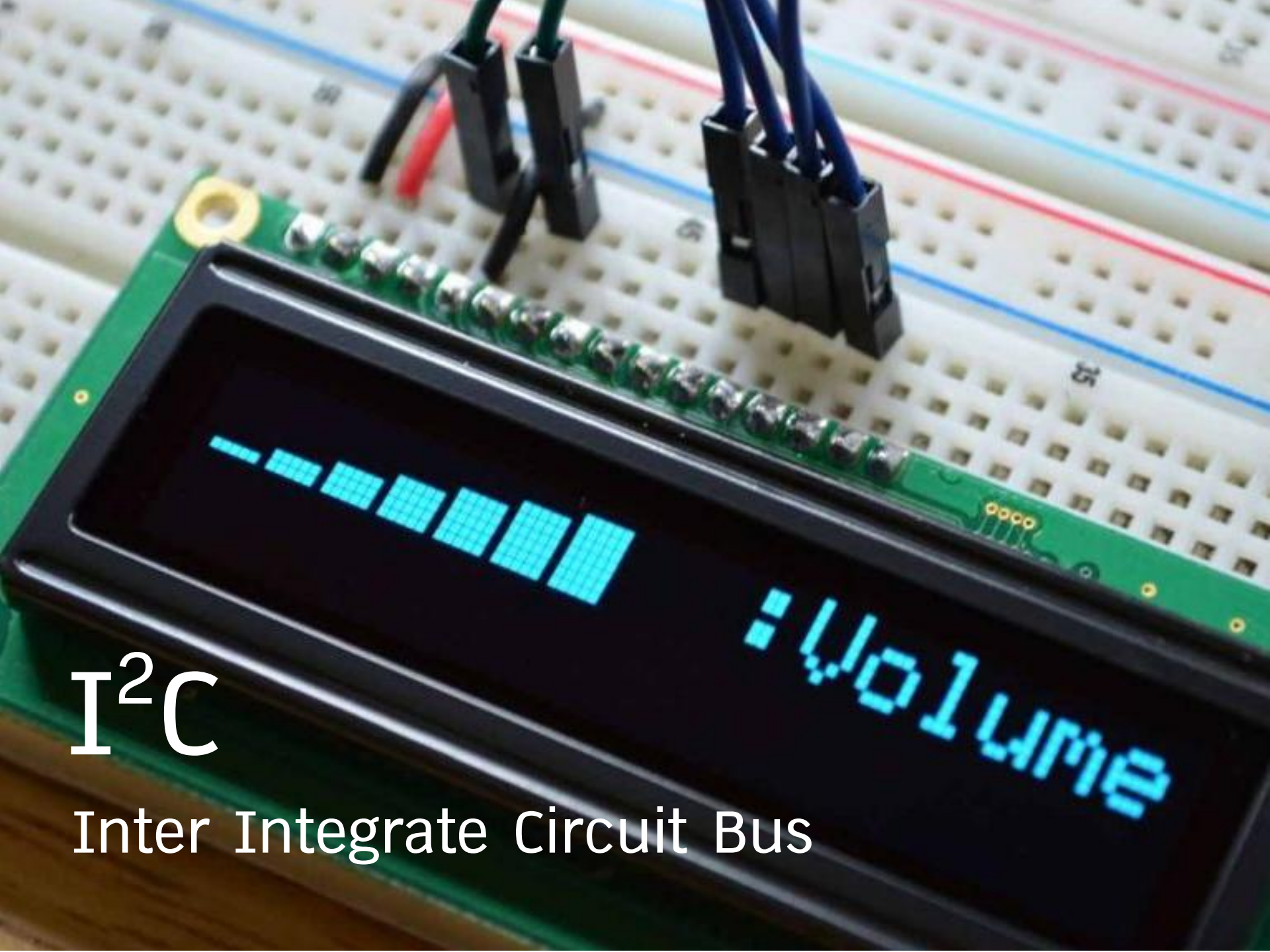
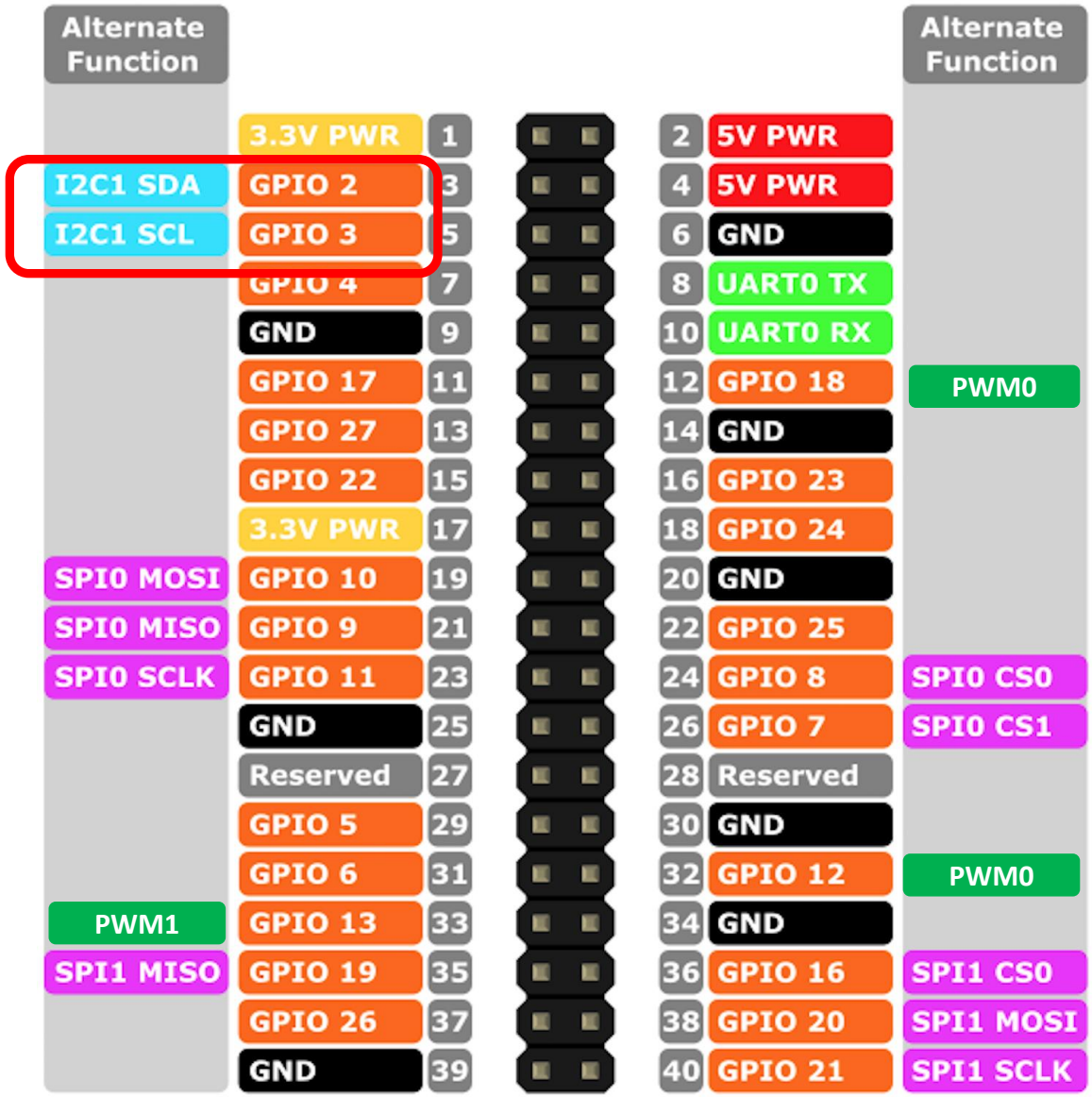

I²C
Inter Integrate Circuit Bus

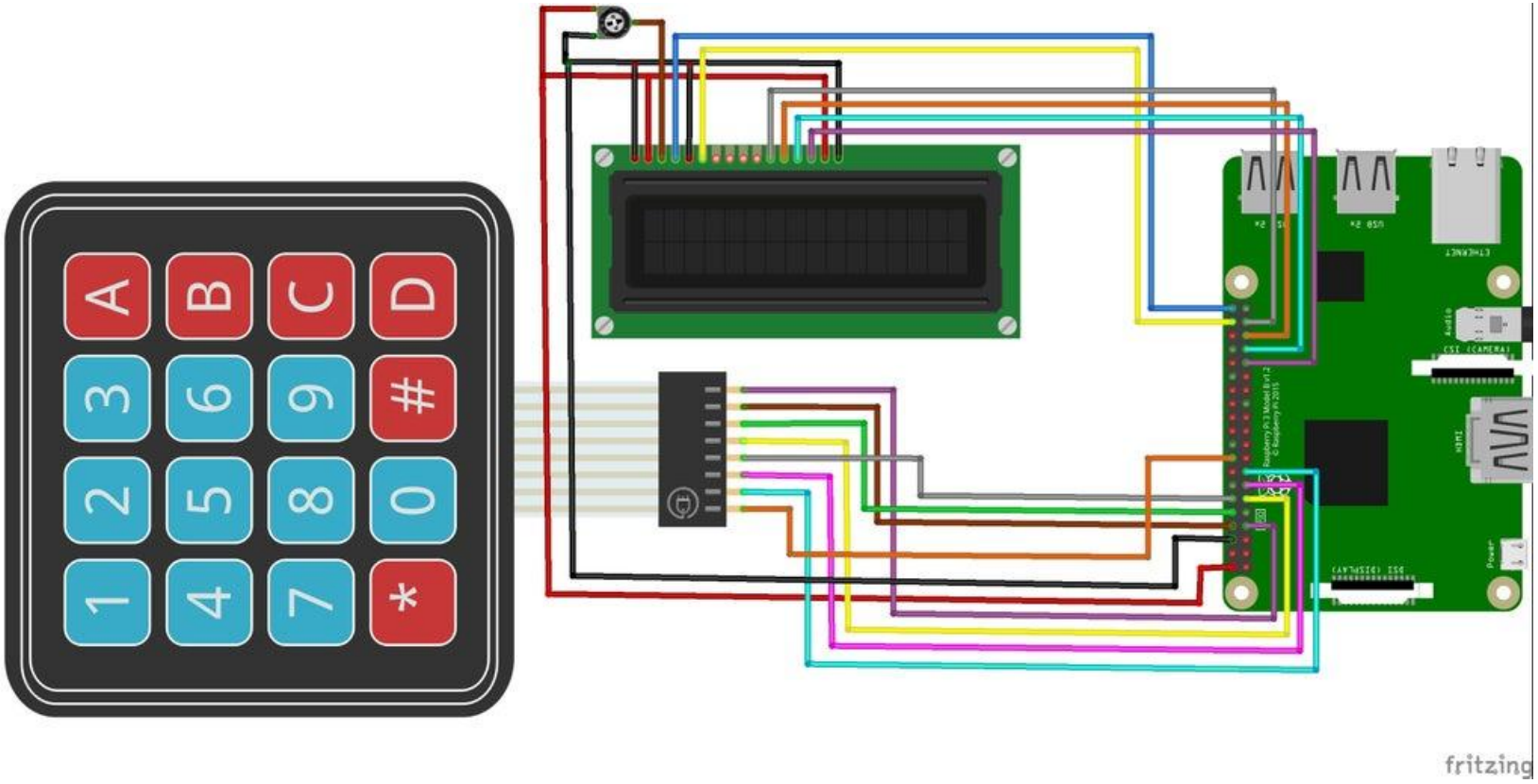
1. Inter Integrate Circuit Bus (I^2C)



ที่มาของปัญหา

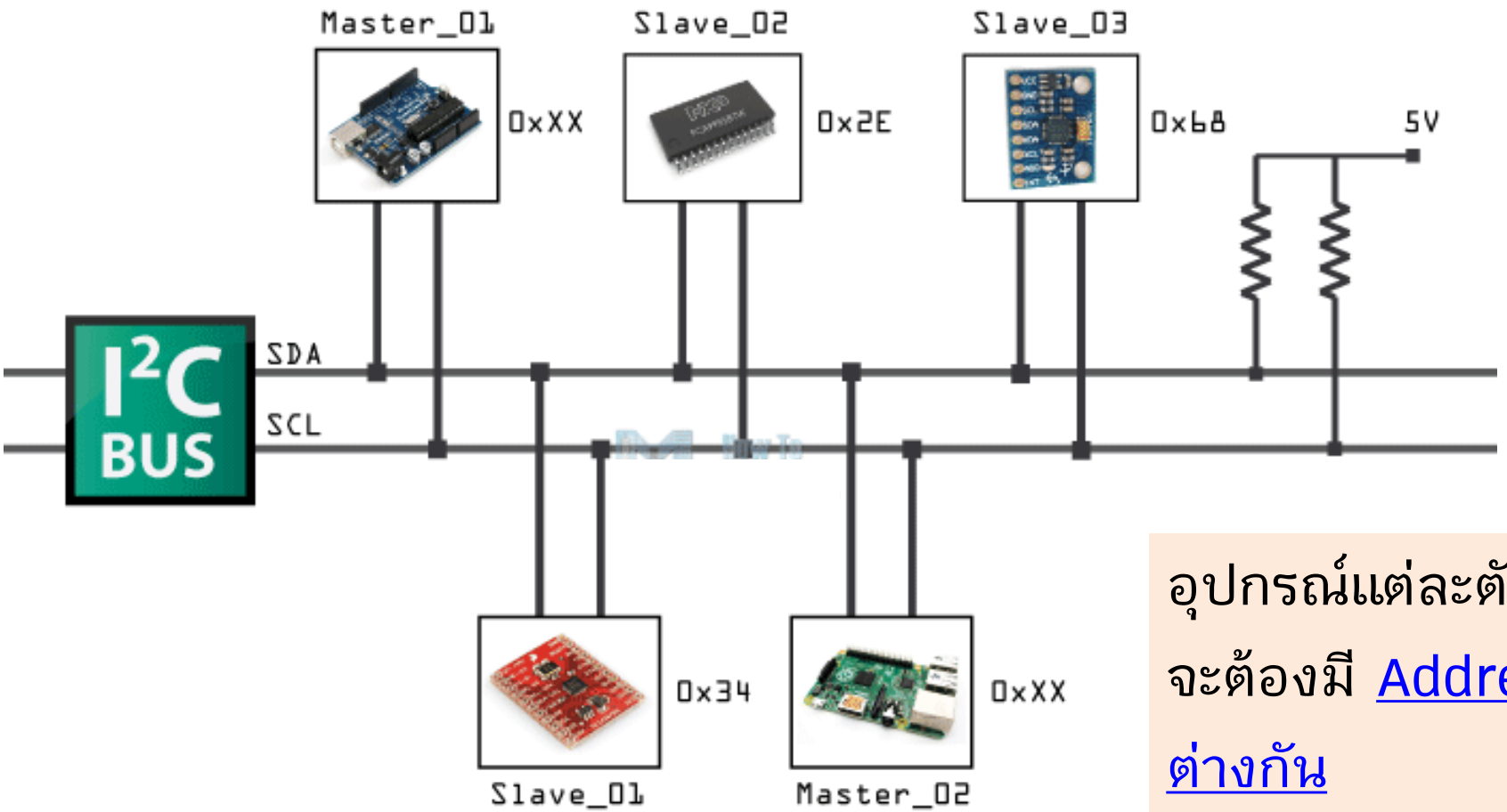
พอต่ออุปกรณ์เยอะๆ จะพบว่า

พอร์ต GPIO ไม่พอใช้



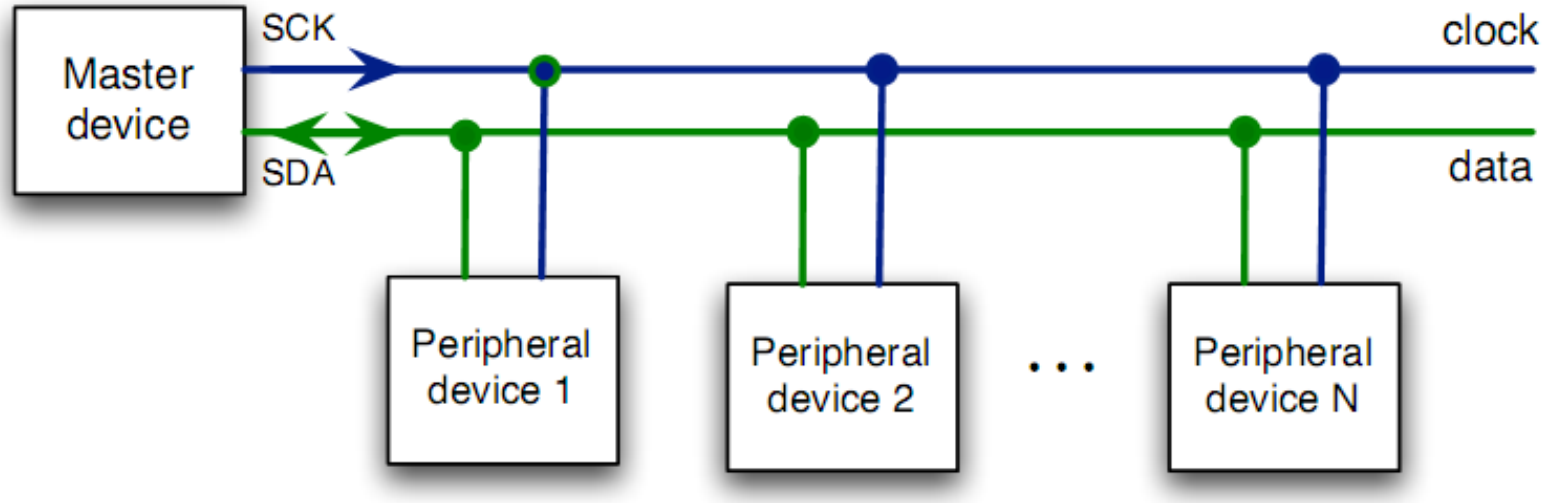
แนวทางการแก้ปัญหา

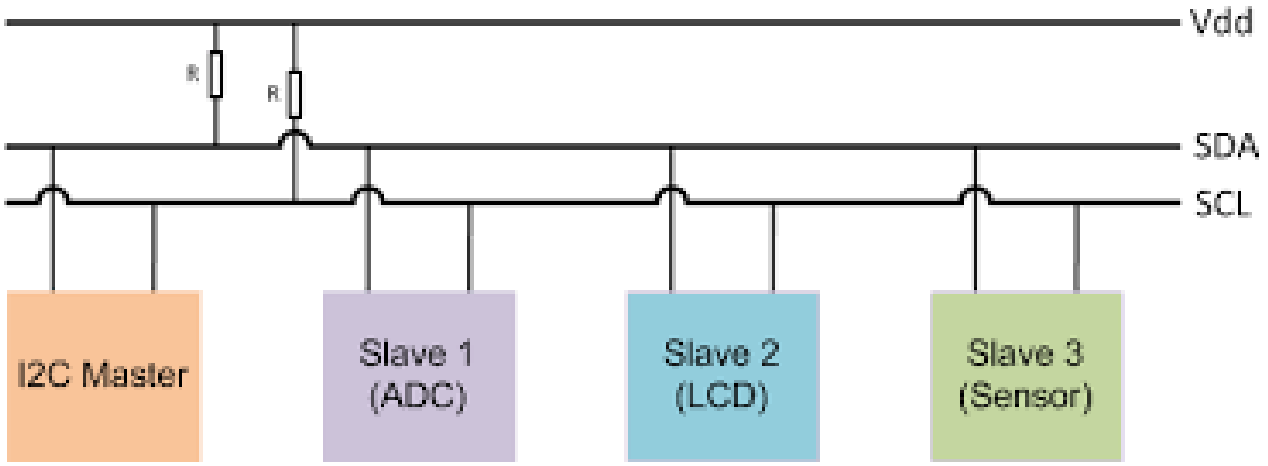
ใช้แค่ 2 พอร์ต แล้วต่อพ่วงอุปกรณ์ โดยมี Master & Slave



อุปกรณ์แต่ละตัวจะต้องมี Address ต่างกัน

I^2C , “Two Wire”


Master device
SCK
SDA
clock
data
Peripheral device 1
Peripheral device 2
Peripheral device N

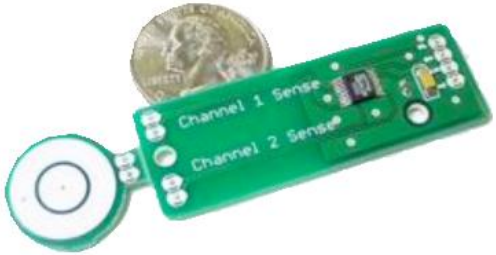

Vdd
R
R
SDA
SCL
I2C Master
Slave 1 (ADC)
Slave 2 (LCD)
Slave 3 (Sensor)

I^2C , “Two Wire”

- I^2C Bus ย่อมาจาก Inter Integrate Circuit Bus เป็นการสื่อสารอนุกรม แบบซิงค์โครนัส (Synchronous) เพื่อใช้ติดต่อสื่อสาร ระหว่าง ไมโครคอนโทรลเลอร์ (MCU) กับอุปกรณ์ภายนอก

- ถูกพัฒนาขึ้นโดยบริษัท Philips Semiconductors โดยใช้สายสัญญาณเพียง 2 เส้นเท่านั้น คือ Serial Data (SDA) และสาย Serial Clock (SCL) ซึ่งสามารถ เชื่อมต่ออุปกรณ์จำนวนหลายๆ ตัว เข้าด้วยกันได้ ทำให้ MCU ใช้พอร์ตเพียง 2 พอร์ตเท่านั้น

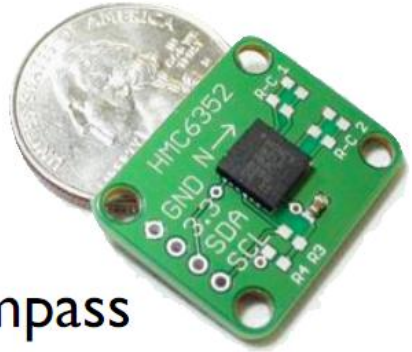
ตัวอย่าง I²C devices



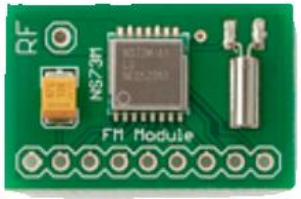
touch sensor



non-volatile memory



compass



fm transmitter



LCD display

And many others

(gyros,keyboards, motors,...)



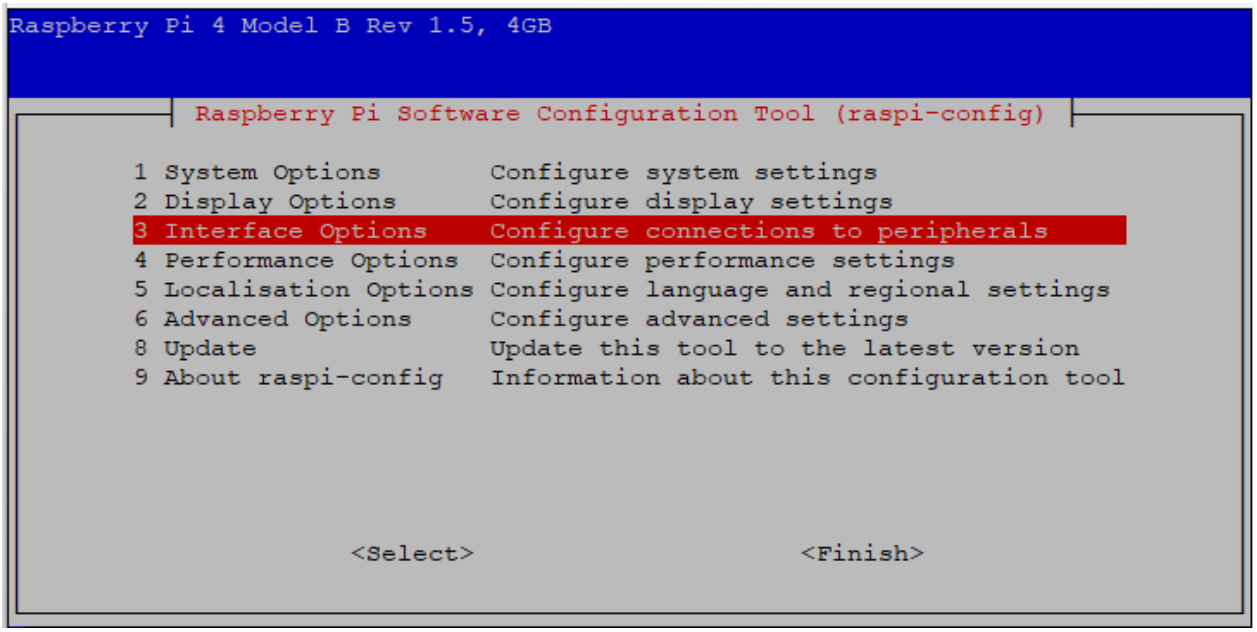
temperature & humidity sensor

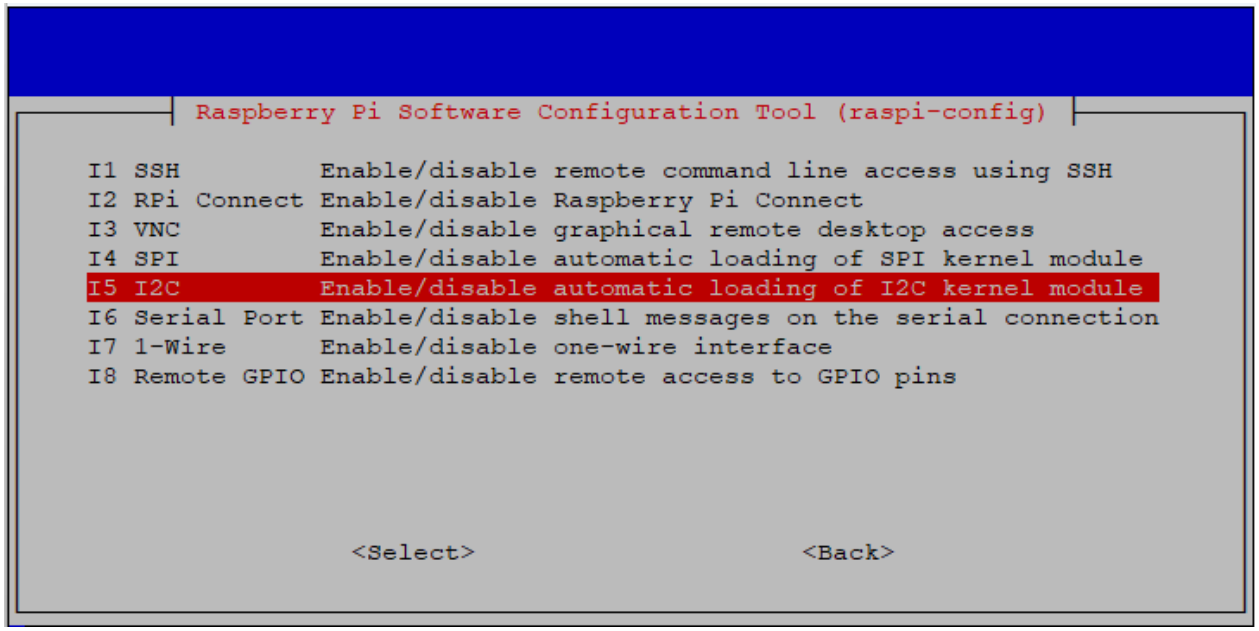
I²C on Raspberry Pi

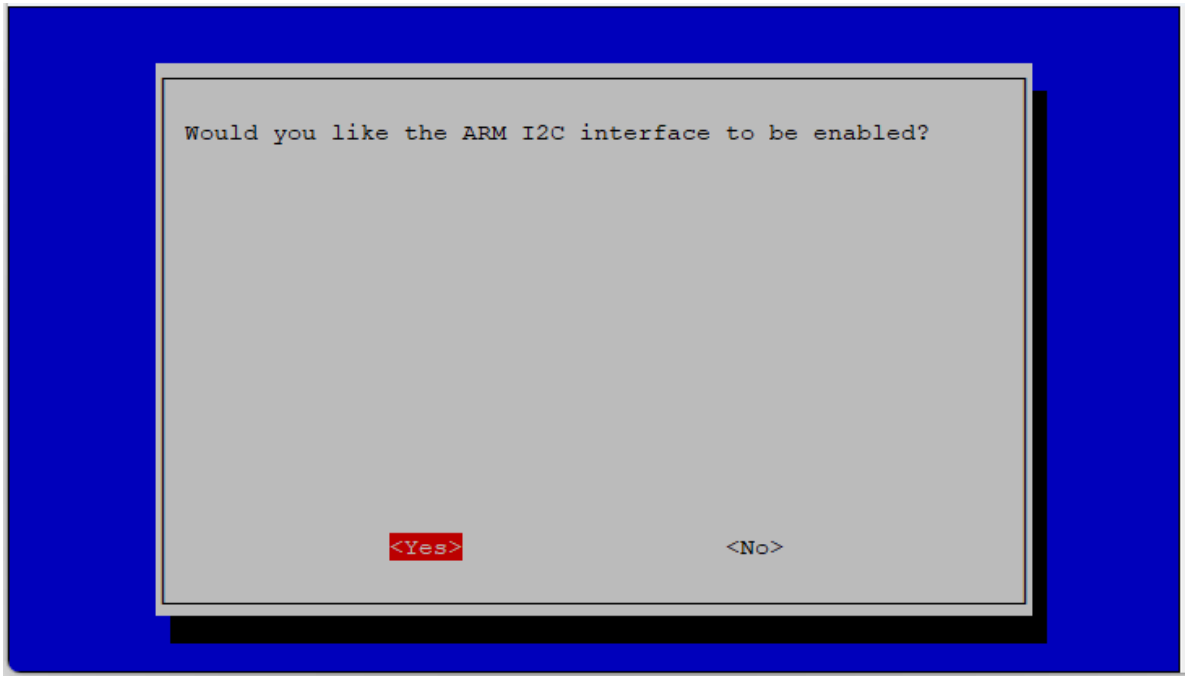
For Raspberry Pi OS, enable the I²C driver as follows:

1. Run `sudo raspi-config`
2. Use the down arrow to select `3 Interface Options`
3. Arrow down to `I5 I2C`
4. Select **yes** when it asks you to enable I2C
5. Also select **yes** when it asks about automatically loading the kernel module
6. Use the right arrow to select the **<Finish>** button

sudo raspi-config

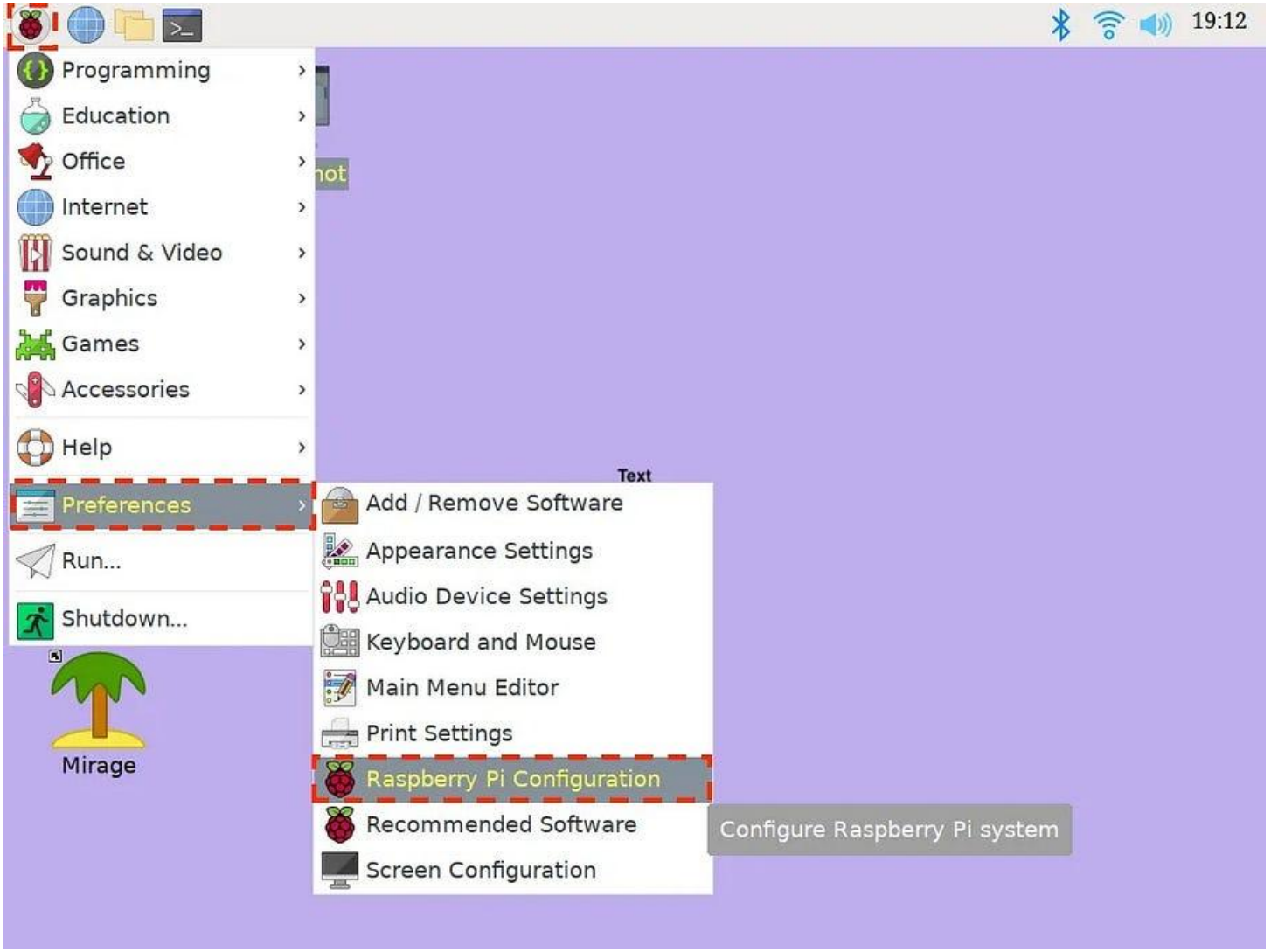


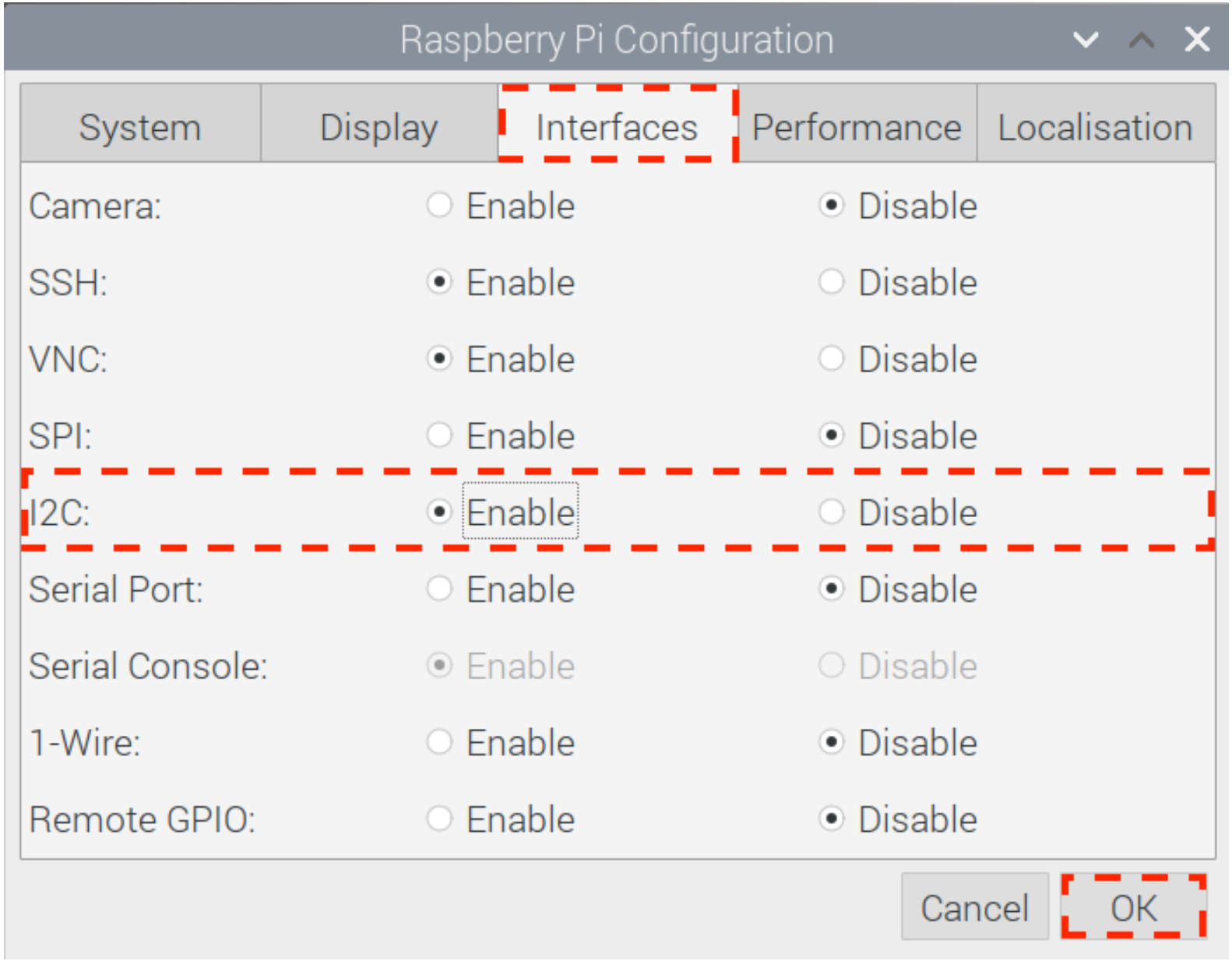



Would you like the ARM I2C interface to be enabled?
<Yes>
<No>


The ARM I2C interface is enabled
<Ok>

กรณีใช้ graphical user interface (GUI)




Raspberry Pi Configuration
System
Display
Interfaces
Performance
Localisation
Camera: Enable Disable
SSH: Enable Disable
VNC: Enable Disable
SPI: Enable Disable
I2C: Enable Disable
Serial Port: Enable Disable
Serial Console: Enable Disable
1-Wire: Enable Disable
Remote GPIO: Enable Disable
Cancel
OK

I²C on Raspberry Pi

i2c-tools เป็น เครื่องมือสำหรับทำงานกับบัส I²C บน Linux ที่ช่วยให้

- สแกนหาอุปกรณ์ I²C ที่เชื่อมต่อกับบอร์ด (i2cdetect)
- อ่าน/เขียนข้อมูลไปยังอุปกรณ์ I²C (i2cget, i2cset ,i2cdump)
- ทำ I2C transactions (i2ctransfer)

ติดตั้ง **$sudo apt-get update**
$sudo apt-get install i2c-tools

```
pi@obeone:~ $ sudo apt-get install i2c-tools
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libi2c0 read-edid
```

คำสั่งของ i2ctool

I2cset เขียนข้อมูลไปยัง I2C device

```
#  เขียน byte เขียนค่า 0xFF ไปยัง register 0x00
     i2cset -y 1 0x48 0x00 0xFF
#  เขียน word (2 bytes)
     i2cset -y 1 0x48 0x00 0x1234 w
#  เขียนหลาย bytes
     i2cset -y 1 0x48 0x00 0x01 0x02 0x03 I
```

i2cget อ่านข้อมูลจาก I2C device

```
# อ่าน byte จาก register
     i2cget -y 1 0x48 0x00
#  อ่าน word (2 bytes)
     i2cget -y 1 0x48 0x00 w
#  อ่าน block data
     i2cget -y 1 0x48 0x00 i
```

i2ctransfer สำหรับการทำ I2C transaction

รูปแบบ descriptor

```
w<length>[@address] - เขียน (write)
r<length>[@address] - อ่าน (read)

#  เขียน 1 byte ไปยัง address 0x50
     i2ctransfer -y 1 w1@0x50 0x12#

#  เขียนหลาย byte
     i2ctransfer -y 1 w3@0x50 0x01 0x02 0x03

#  อ่าน 1 byte จาก address 0x50
     i2ctransfer -y 1 r1@0x50

#  อ่าน 4 bytes
     i2ctransfer -y 1 r4@0x50
```

โหมดข้อมูล (สำหรับ i2cget/i2cset/i2cdump)

```
b: byte (8-bit) – default
w: word (16-bit)
c: consecutive byte
i: I2C/SMBus block data
s: SMBus block data
```

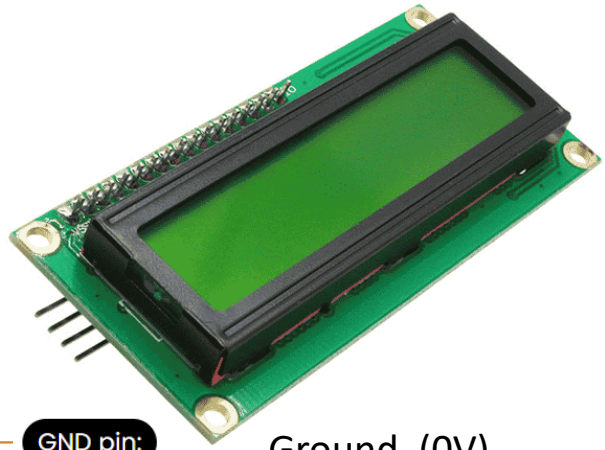
Flags ทั่วไป:

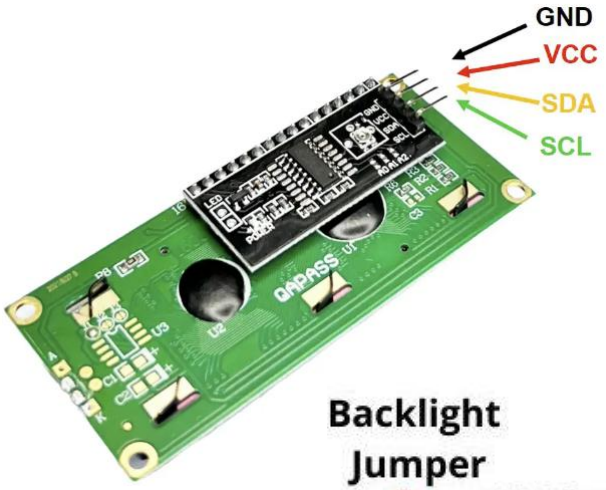
- y: ข้ามการถามยืนยัน (assume yes)
- f: บังคับทำงานแม้อุปกรณ์ไม่รองรับ
- r: อ่านอย่างเดียว (สำหรับ i2cdetect)

I^2C LCD

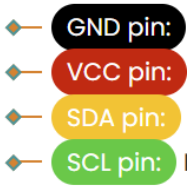


1. I²C Liquid Crystal Display : LCD ขนาด 16x2



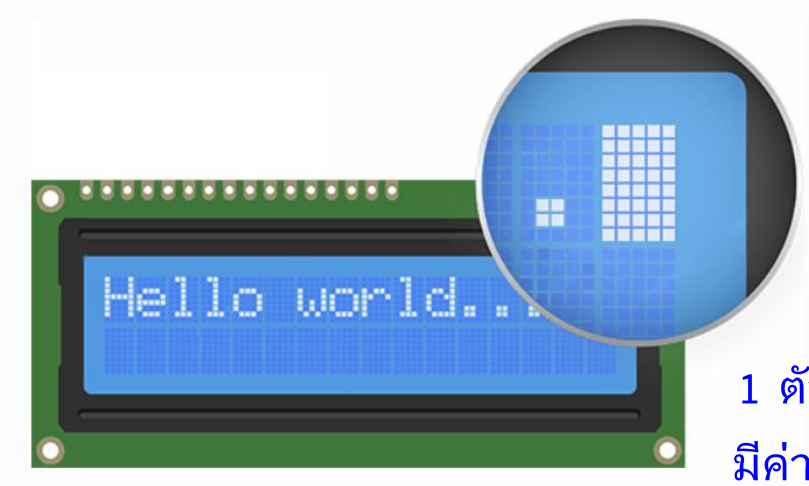


I2C โมดูลสำหรับ LCD ตัวนี้คือ Chip PCF8574



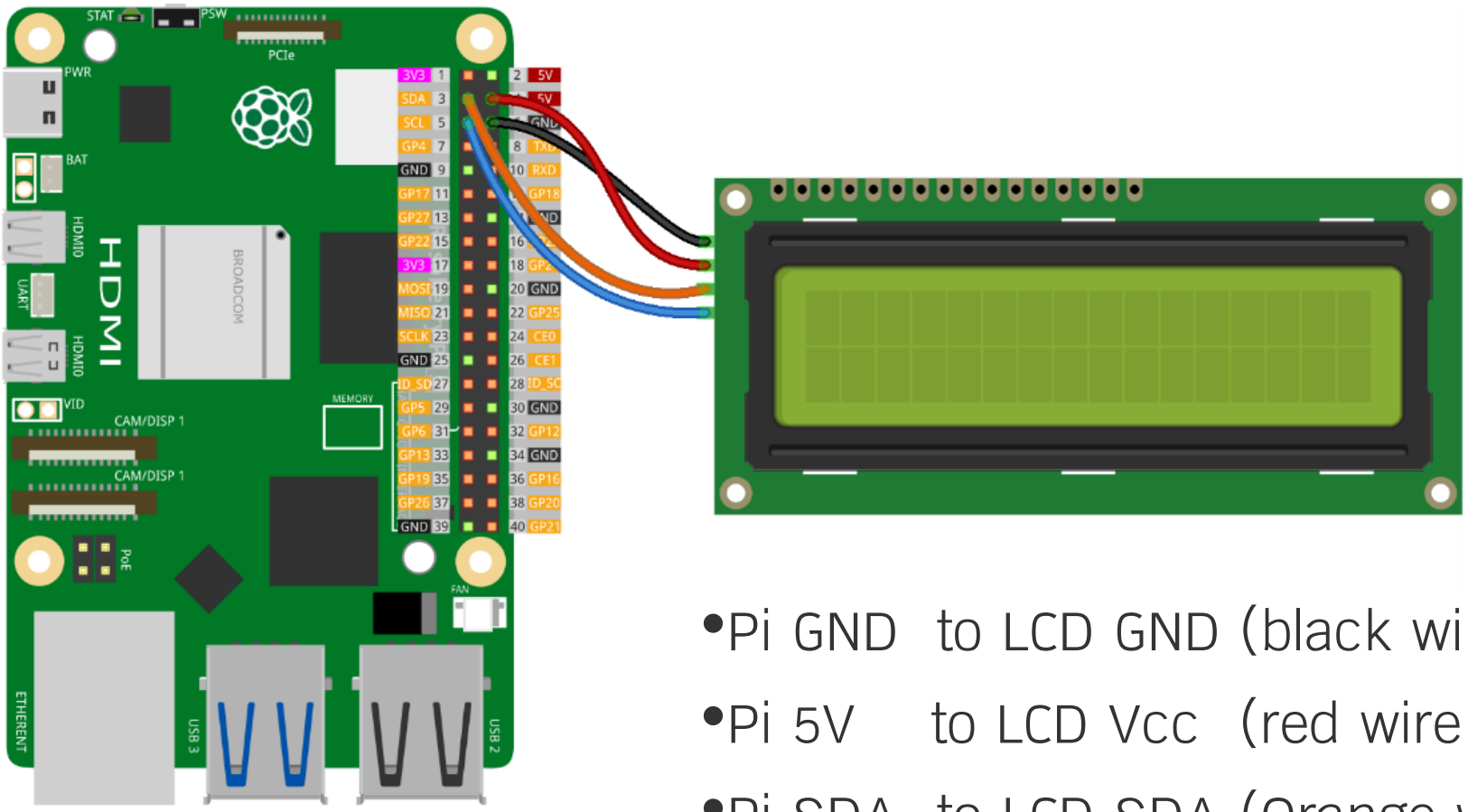
Ground (0V)
Power Supply +5 V
I2C Data Signal
I2C Clock Signal





1 ตัวอักษร
มีค่า 5x8 Pixels

การทดลองที่ 1. ต่อ วงจร I^2C LCD



- Pi GND to LCD GND (black wire)
- Pi 5V to LCD Vcc (red wire)
- Pi SDA to LCD SDA (Orange wire)
- Pi SCL to LCD SCL (blue wire)

ตัวอย่าง การสแกนหาอุปกรณ์ I²C ที่เชื่อมต่อกับบอร์ด

$sudo i2cdetect -y 1

Note สแกนบัส i2c หมายเลข 1 เพื่อหาว่ามีอุปกรณ์อะไรเชื่อมอยู่ ซึ่งใน raspberry 4 มีแค่ bus หมายเลข 1

คำอธิบาย **-y :** บังคับให้รันคำสั่งโดย ไม่ถามยืนยัน (yes/no) **1** คือ bus หมายเลข 1

The `-y` option assumes we answer yes to any prompts. `1` is the I2C bus number. It's always `1` on the Raspberry Pi 3 or 4. We expect to see the output like this:

```
pi@obeone:~ $ sudo i2cdetect -y 1
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                         -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- 27 -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
pi@obeone:~ $
```

จะพบว่า Raspi เจอ อุปกรณ์หมายเลข Address เท่ากับ 27 ซึ่งก็คือจอ I²C LCD

2. Function ที่ใช้ในการควบคุมจอ LCD

Raspi มี Library มาตรฐานสำหรับการเชื่อมต่อจอ LCD ชื่อ RPLCD

https://pypi.org/project/RPLCD/

Initializing the LCD

```
from RPLCD.i2c import CharLCD

lcd = CharLCD('PCF8574', 0x27)
```

หรือ

```
lcd = CharLCD(i2c_expander='PCF8574', address=0x27, port=1,
              cols=16, rows=2, dotsize=8,
              charmap='A00',
              auto_linebreaks=True,
              backlight_enabled=True)
```

Port = 1 คือ I2C bus1 (raspi มีแค่ bus1) ,

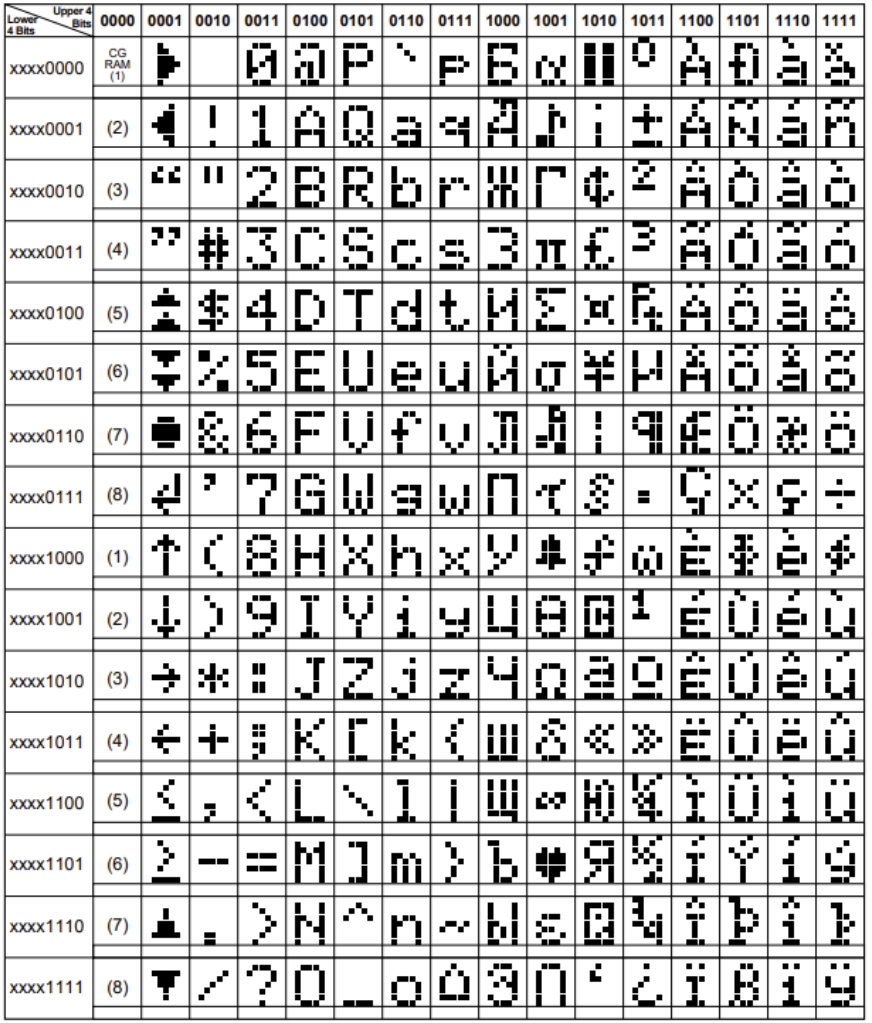
charmap='A00' คือ ASCII standard ถ้า charmap='A02' คือ ชุดอักษรอื่นๆ

Table 4 Correspondence between Character Codes and Character Patterns (ROM Code: A00)



ASCII standard

Table 4 Correspondence between Character Codes and Character Patterns (ROM Code: A02)



ชุดอักษรอื่นๆ

คำสั่ง	ความหมาย
`lcd.clear()`	Clears the display
`lcd.crlf()`	ขึ้นบรรทัดใหม่ , to write line feed (\n) or carriage return (\r) characters to the display.
`lcd.write_string('string')`	Regular text can be written to the LCD, It accepts unicode strings
`lcd.cursor_pos = (row, column)`	The cursor position can be set by assigning a (row, col)

ติดตั้ง Library

```
$sudo apt-get install python3-pip
$sudo pip3 install RPLCD --break-system-packages
$sudo pip3 install smbus3 --break-system-packages
```

ตัวอย่าง Code

```
from RPLCD.i2c import CharLCD
import time

# สร้างออบเจ็กต์ LCD
lcd = CharLCD(
    i2c_expander='PCF8574',
    address=0x27,
    port=1,
    cols=16,
    rows=2,
    charmap='A00',
    auto_linebreaks=True,
    backlight_enabled=True
)
# แสดงข้อความ
lcd.write_string("Hello, Raspberry")
lcd.crlf()  # ขึ้นบรรทัดใหม่
lcd.write_string("I2C LCD Ready!")

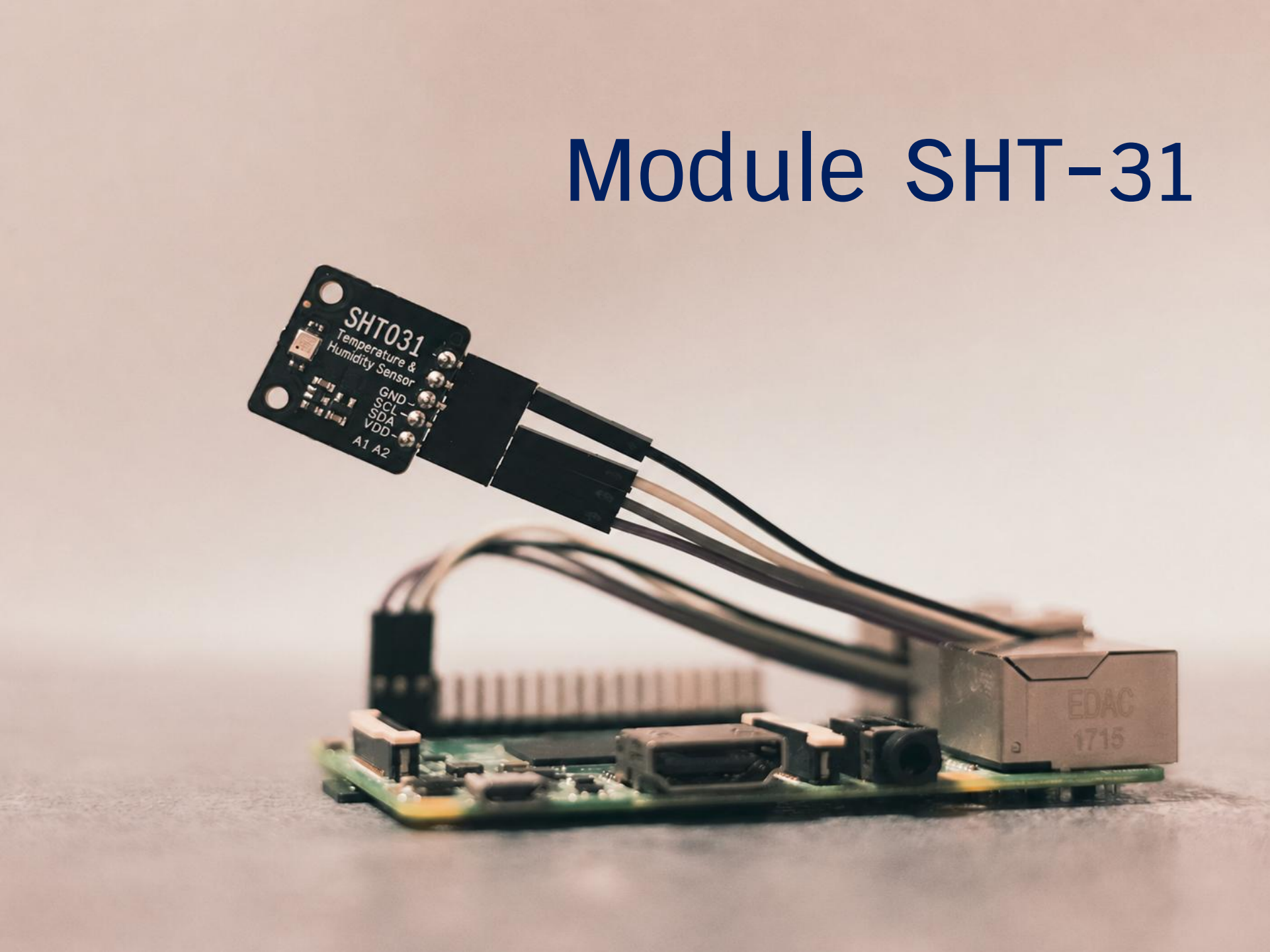
time.sleep(5)

lcd.clear() # ล้างหน้าจอ  สดงข้อความใหม่
lcd.write_string("Line 1 Example")
lcd.crlf()
lcd.write_string("Temp: 25.5 C")
```





Module SHT-31



Module SHT-31

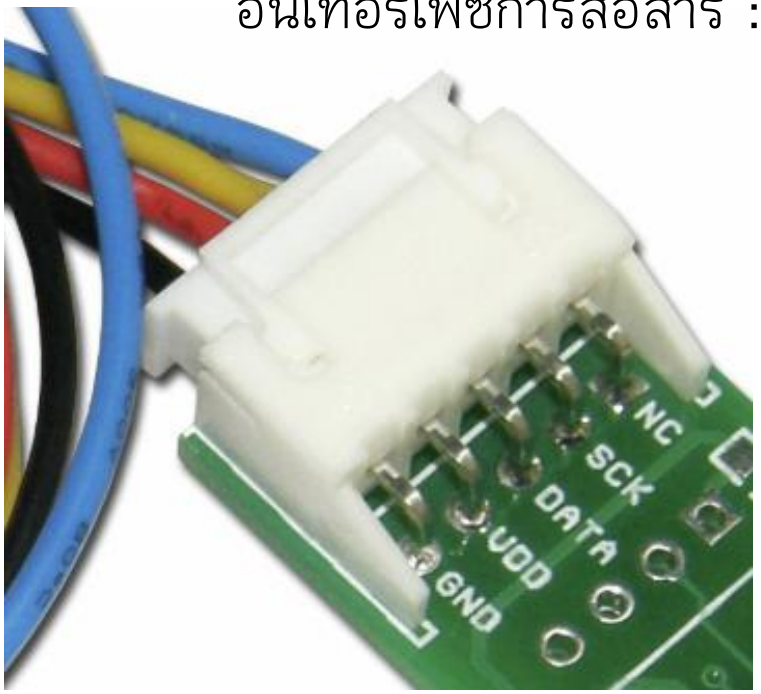
SHT31 คือ เซนเซอร์วัดอุณหภูมิและความชื้นสัมพัทธ์ ที่ผลิตโดยบริษัท Sensirion (สวิตเซอร์แลนด์) วัดได้ทั้งอุณหภูมิและความชื้น ในตัวเดียว

ช่วงการวัด

ความชื้น: 0 – 100 %RH (ความแม่นยำ ±2%RH)

อุณหภูมิ: -40 °C ถึง +125 °C (ความแม่นยำ ±0.3 °C)

อินเทอร์เฟซการสื่อสาร : I²C





ข้อมูลจาก Datasheet : SHT31

Device Address

SHT3x-DIS	I2C Address in Hex. representation	Condition
I2C address A	0x44 (default)	ADDR (pin 2) connected to VSS
I2C address B	0x45	ADDR (pin 2) connected to VDD

Table 7 I2C device address

Measurement Commands for Single Shot Data Acquisition Mode

Datasheet SHT3x-DIS

Condition		Hex. code	
Repeatability	Clock stretching	MSB	LSB
High	enabled	0x2C	06
Medium			0D
Low			10
High	disabled	0x24	00
Medium			0B
Low			16
e.g. 0x2C06: high repeatability measurement with clock stretching enabled			

ข้อมูลที่ IC ส่งกลับมา ขนาด 6 byte

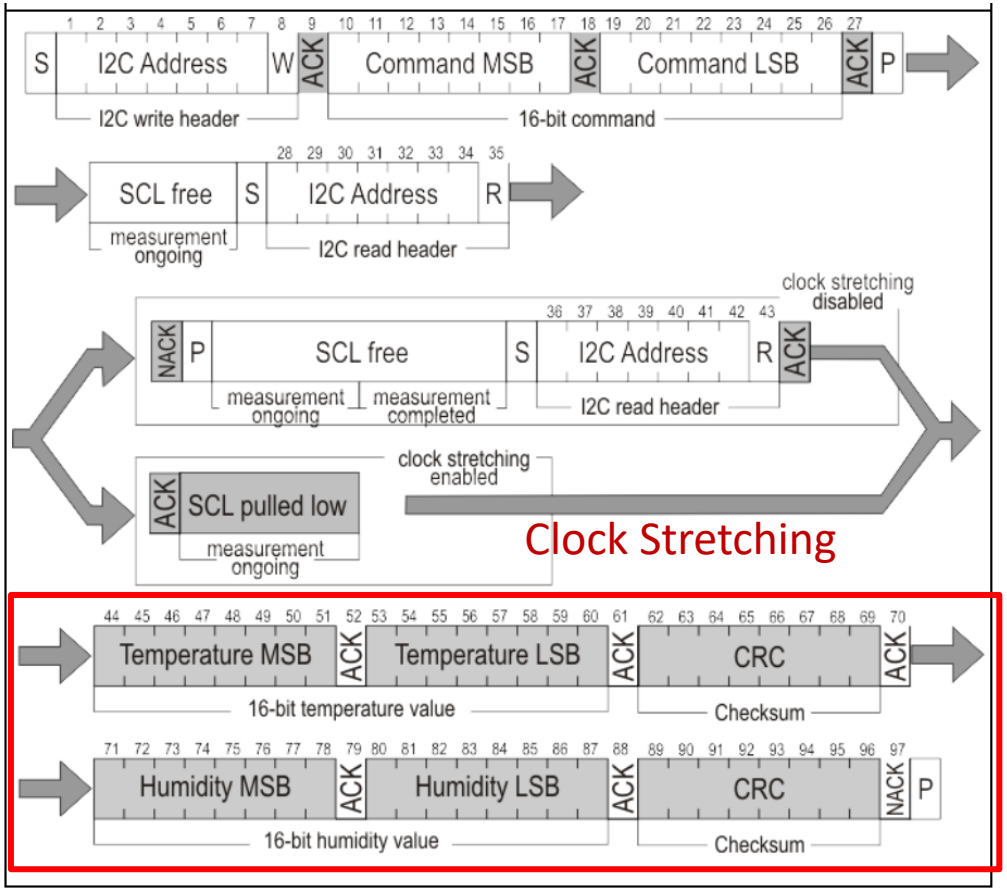
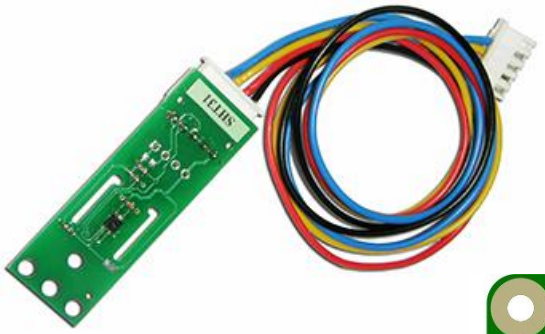


Table 8 Measurement commands in single shot mode (Clear blocks are controlled by the microcontroller, grey blocks by the sensor)

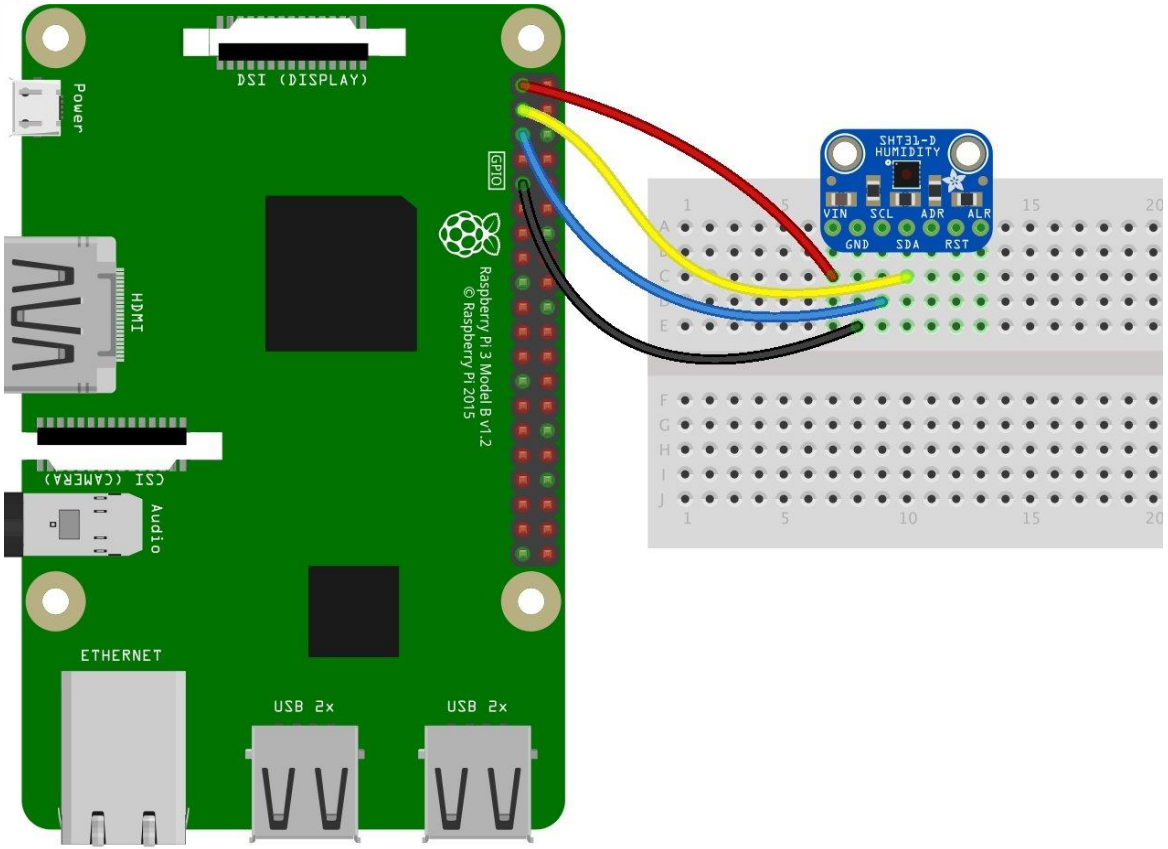
Note!! Clock Stretching

- โดยปกติ การสื่อสารแบบ I2C , ตัวอุปกรณ์ master จะเป็นผู้ควบคุมสัญญาณ Clock ในการสื่อสาร เพื่อให้ Synchronize กับอุปกรณ์ master (Synchronize แปลว่า ตรงกัน)
- บางครั้ง อุปกรณ์ Slave ต้องการเวลาในการทำงาน หรือประมวลผล เช่น sensor ต้องใช้เวลาในการแปลงค่าอุณหภูมิ เป็นข้อมูล , memory ต้องการเวลาในการอ่าน เขียนข้อมูล
- ดังนั้น บางอุปกรณ์ จึงยอมให้อุปกรณ์ Slave นั้นๆ กำหนดสัญญาณ clock ให้เป็น LOW เพื่อหยุดการสื่อสาร เมื่ออุปกรณ์ทำงานเสร็จ ถึงจะปล่อยให้ clock ทำงานปกติ จากนั้น การสื่อสารถึงจะดำเนินการต่อไปได้ตามปกติ
- Clock Stretching คือ การยอมให้ Slave ควบคุมการสื่อสารได้

การทดลองที่ 2. Temp & Humidity Sensor : SHT31



- Pi 3V3 to sensor VIN (red wire)
- Pi GND to sensor GND (black wire)
- Pi SDA to sensor SDA (yellow wire)
- Pi SCL to sensor SCL (blue wire)



ทบทวน คำสั่งของ i2ctool

I2cset เขียนข้อมูลไปยัง I2C device

```
#  เขียน byte เขียนค่า 0xFF ไปยัง register 0x00
     i2cset -y 1 0x48 0x00 0xFF
#  เขียน word (2 bytes)
     i2cset -y 1 0x48 0x00 0x1234 w
#  เขียนหลาย bytes
     i2cset -y 1 0x48 0x00 0x01 0x02 0x03 I
```

i2cget อ่านข้อมูลจาก I2C device

```
# อ่าน byte จาก register
     i2cget -y 1 0x48 0x00
#  อ่าน word (2 bytes)
     i2cget -y 1 0x48 0x00 w
#  อ่าน block data
     i2cget -y 1 0x48 0x00 i
```

i2ctransfer สำหรับการทำ I2C transaction

รูปแบบ descriptor

```
w<length>[@address] - เขียน (write)
r<length>[@address] - อ่าน (read)

# เขียน 1 byte ไปยัง address 0x50
     i2ctransfer -y 1 w1@0x50 0x12#

# เขียนหลาย byte
     i2ctransfer -y 1 w3@0x50 0x01 0x02 0x03

# อ่าน 1 byte จาก address 0x50
     i2ctransfer -y 1 r1@0x50

# อ่าน 4 bytes
     i2ctransfer -y 1 r4@0x50
```

โหมดข้อมูล (สำหรับ i2cget/i2cset/i2cdump)

```
b: byte (8-bit) – default
w: word (16-bit)
c: consecutive byte
i: I2C/SMBus block data
s: SMBus block data
```

Flags ทั่วไป:

- y: ข้ามการถามยืนยัน (assume yes)
- f: บังคับทำงานแม้อุปกรณ์ไม่รองรับ
- r: อ่านอย่างเดียว (สำหรับ i2cdetect)

ตัวอย่างการใช้ I2C tool

- `sudo i2cdetect -y 1`

```
pi@obeone:~ $ sudo i2cdetect -y 1
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                         -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- 44 -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
pi@obeone:~ $
```

- `sudo i2cset -y 1 0x44 0x24 0x00`
- `sudo i2ctransfer -y 1 r6@0x44`

Note

IC ใช้เวลาในการแปลงข้อมูล 15 ms (High repeatability)

```
pi@obeone: ~
pi@obeone:~ $ sudo i2cset -y 1 0x44 0x24 0x00
pi@obeone:~ $ sudo i2ctransfer -y 1 r6@0x44
0x6a 0x33 0xac 0x8b 0x3c 0x00
pi@obeone:~ $
```

สูตรการคำนวณการแปลงข้อมูล

จากตัวอย่าง ค่าที่รับได้ คือ 0x6a 0x33 0xac 0x8b 0x3c 0x00

จะได้ว่า

Temperature

MSB = 0x6a , LSB = 0x33 , CRC = 0xac

Humidity

MSB = 0x8b , LSB = 0x3c , CRC = 0x00

แปลงฐาน 16 เป็นค่าจริง

MSB = 0x6a , LSB = 0x33 = 256(6*16+10) + (3*16+3) = 27,187

MSB = 0x8b , LSB = 0x3c = 256(8*16+11) + (3*16+12) = 35,644

สูตรการคำนวณการแปลงข้อมูล

voltage effects. Converting those raw values into a physical scale can be achieved using the following formulas.

Relative humidity conversion formula (result in %RH):

$$RH = 100 \cdot \frac{S_{RH}}{2^{16} - 1}$$

ดังนั้น จากค่าในหน้าที่แล้ว อ่านค่าได้เป็น

100*(35,644 / 65535) = 54.41 %

Temperature conversion formula (result in °C & °F):

$$T[°C] = -45 + 175 \cdot \frac{S_T}{2^{16} - 1}$$

-45 + 175*(27,187 / 65535) = 27.64 Celcius

$$T[°F] = -49 + 315 \cdot \frac{S_T}{2^{16} - 1}$$

S_{RH} and S_T denote the raw sensor output for humidity and temperature, respectively. The formulas work only correctly when S_{RH} and S_T are used in decimal representation.

Library smbus3

- smbus3 คือ Library ของ Python สำหรับติดต่อสื่อสารผ่านโปรโตคอล I²C (Inter-Integrated Circuit)
- รองรับการอ่านเขียนข้อมูลกับอุปกรณ์ I²C บน Linux (เช่น Raspberry Pi)

ติดตั้ง Library

```
$sudo apt-get install python3-pip
$sudo pip3 install smbus3 --break-system-packages
```

Library smbus3

- สมมติว่าอุปกรณ์ I²C มี address เท่ากับ 0x48
- เขียน 1 byte
  - `bus.write_byte(0x48, 0x01)`
  - `bus.i2c_wr(0x48, [0x01])`

    # ส่งค่า 0x01 ไปที่อุปกรณ์
- เขียน ระบุ address
  - `bus.write_byte_data(0x48, 0x10, 0x55)`

    # เขียนค่า 0x55 ลงใน register 0x10 ไปที่อุปกรณ์
- เขียนหลาย byte
  - `data = [0x01, 0x02, 0x03, 0x04]`
  - `bus.write_i2c_block_data(0x48,0x10, data)`
  - `bus.i2c_wr(0x48, [0x01, 0x02, 0x03, 0x04])`

Library smbus3

- สมมติว่าอุปกรณ์ I²C มี address เท่ากับ 0x48
- อ่าน 1 byte
  - `value = bus.read_byte(0x48)`
- อ่านจากการระบุ address
  - `value = bus.read_byte_data(0x48,0x10)`

    `# อ่านค่าจาก register 0x10`
- อ่านหลาย byte
  - `value = bus.read_i2c_block_data(0x48, 0x10, 4)`
  - `value = bus.i2c_rd(0x48, 4)`

    `# อ่าน 4 ไบต์ โดยเริ่มอ่านจาก register 0x10`

ตัวอย่าง code

```
import smbus3
import time

# Get I2C bus
bus = smbus3.SMBus(1)

# SHT31 address, 0x44(68)
# Send measurement command, 0x2C(44)
# 0x06(06)High repeatability measurement
bus.i2c_wr(0x44,[0x2C,0x06])
time.sleep(0.5)

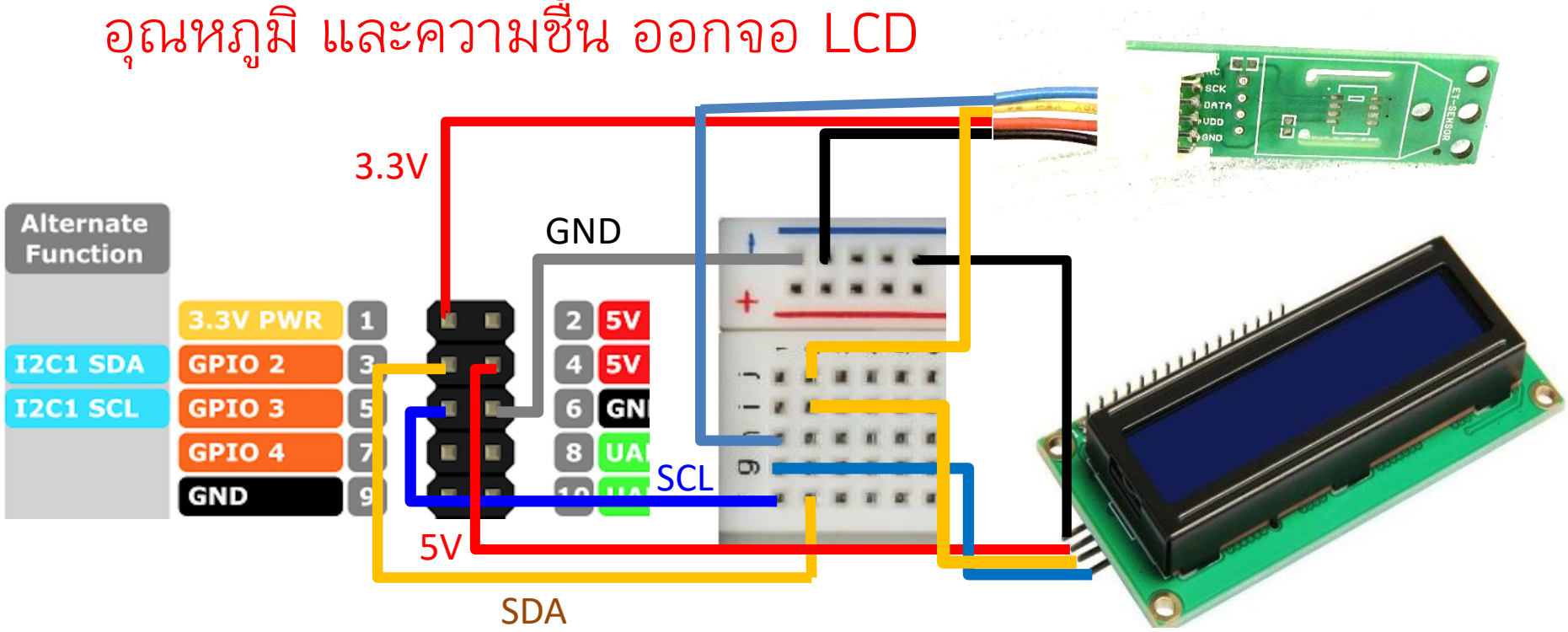
# SHT31 address, 0x44(68)
# Read data back from 0x00(00), 6 bytes
# Temp MSB, Temp LSB, Temp CRC, Humidity MSB, Humidity LSB, Humidity CRC
msg  = bus.i2c_rd(0x44, 6)
data = bytes(msg)
bus.close

# Convert the data
temp = data[0] * 256 + data[1]
cTemp = -45 + (175 * temp / 65535.0)
humidity = 100 * (data[3] * 256 + data[4]) / 65535.0

# Output data to screen
print( "Temperature in Celsius is : %.2f C" %cTemp)
print( "Relative Humidity is : %.2f %%RH" %humidity)
```

แบบฝึกหัด

- ให้ต่อวงจร รวม 2 อุปกรณ์ และ เขียนโปรแกรมให้แสดงผลอุณหภูมิ และความชื้น ออกจอ LCD



Hint ถ้ามีตัวเลข จะให้แสดงออกจอ LCD ให้เปลี่ยนเป็น String เสียก่อน

```
a = 24
lcd.write_string("Value: " + str(a))
```


The End